SymPy Bug Report

1 Bug 25: `solveset` returns a false solution for `acsc(x) = 3*pi/2`

1.1 Minimal Reproducer

```
from sympy import Eq, S, acsc, pi, simplify, solveset, symbols

x = symbols("x")
sol = solveset(Eq(acsc(x), 3*pi/2), x, S.Complexes)
print(sol)
for candidate in sol:
    print(candidate, acsc(candidate), simplify(acsc(candidate) - 3*pi/2))
```

1.2 Observed Behavior

```
{-1}
-1 -pi/2 -2*pi
```

SymPy returns the finite solution set $\{-1\}$. Direct substitution shows that the returned value does not satisfy the original equation: under SymPy's principal branch, $\operatorname{acsc}(-1) = -\pi/2$, so the residual is -2π.

1.3 Expected Behavior

The expected result is $\emptyset$. The value $3\pi/2$ is outside the principal range of acsc, and no complex value x can satisfy the equation $\operatorname{acsc}(x) = 3\pi/2$ when acsc is interpreted as SymPy's principal inverse cosecant.

1.4 Mathematical Explanation

SymPy's inverse cosecant is the principal inverse function, not a multivalued inverse modulo 2π. For the returned candidate,

$$\operatorname{acsc}(-1) = -\frac{\pi}{2}.$$

Although

$$\csc\left(\frac{3\pi}{2}\right) = -1,$$

this does not imply that $\operatorname{acsc}(-1) = 3\pi/2$. The values $3\pi/2$ and $-\pi/2$ are coterminal for the periodic cosecant function, but only $-\pi/2$ is the principal value returned by $\operatorname{acsc}(-1)$. Therefore applying csc to both sides of $\operatorname{acsc}(x) = 3\pi/2$ is not a reversible transformation, and the candidate -1 is a false solution rather than an equivalent branch representation.

1.5 Source-Level Diagnosis

The diagnosis identifies the immediate source of the false candidate in `_invert_complex` in `sympy/solvers/solveset.py`. The public `solveset` call for $\operatorname{acsc}(x) = 3\pi/2$ over $\mathbb{C}$ reaches `_solveset`, where the equation is represented as `acsc(x) - 3*pi/2 = 0`. The additive constant is stripped, so `_invert_complex` is asked to invert `acsc(x)` against `FiniteSet(3*pi/2)`.

At that point the generic inverse-function branch in `sympy/solvers/solveset.py` applies `acsc(x).inverse()`. The inverse method for `acsc`, defined in `sympy/functions/elementary/trigonometric.py`, returns `csc`. As a result, the inversion step maps $3\pi/2$ to $\csc(3\pi/2) = -1$ without checking that $3\pi/2$ lies in the principal image of acsc.

```
if hasattr(f, 'inverse') and f.inverse() is not None and \
   not isinstance(f, TrigonometricFunction) and \
   not isinstance(f, HyperbolicFunction) and \
   not isinstance(f, exp):
    ...
    return _invert_complex(f.args[0],
                           imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

The specialized trigonometric inversion logic below this generic branch handles periodic branch families for direct trigonometric functions, but the inverse trigonometric function acsc is handled first by the generic inverse mechanism. The diagnosis therefore points to a missing principal-range guard: inverse-function inversion for inverse trigonometric functions should either impose a range condition on the target value or route these functions through dedicated principal-branch handling. For finite right-hand sides, substituting candidate values back into the original equation would also prevent this false finite solution from being returned.

1.6 Additional Failing Instances

The independent verification covers a small family of targets outside the principal acsc range. In each case, SymPy appears to apply csc to the target and return the resulting finite value, but the returned candidate evaluates under the principal acsc branch to either $\pi/2$ or $-\pi/2$, not to the original target. The expected behavior is therefore the same across the family: no returned candidate should have a nonzero substitution residual, and these displayed equations should return $\emptyset$.

Input / Expression	**SymPy behavior**	**Expected behavior**
$\operatorname{acsc}(x) = 3\pi/2$ over $\mathbb{C}$	returns $\{-1\}$; residual -2π	$\emptyset$
$\operatorname{acsc}(x) = 5\pi/2$ over $\mathbb{C}$	returns $\{1\}$; residual -2π	$\emptyset$
$\operatorname{acsc}(x) = -3\pi/2$ over $\mathbb{C}$	returns $\{1\}$; residual 2π	$\emptyset$
$\operatorname{acsc}(x) = 7\pi/2$ over $\mathbb{C}$	returns $\{-1\}$; residual -4π	$\emptyset$
$\operatorname{acsc}(x) = 11\pi/2$ over $\mathbb{C}$	returns $\{-1\}$; residual -6π	$\emptyset$
$\operatorname{acsc}(x) = -5\pi/2$ over $\mathbb{C}$	returns $\{-1\}$; residual 2π	$\emptyset$

1.7 Related Issues Assessment

The best-effort deduplication check found public background material on principal branches of inverse trigonometric functions and SymPy documentation stating that `solveset` results should satisfy the original equation, but it did not find a public issue, pull request, Stack Overflow post, mailing-list thread, or release note for this exact acsc failure. The deduplication verdict is *new instance of a known failure mode*: the general risk of branch-sensitive inverse trigonometric inversion is known, but this exact reproducible `solveset` counterexample is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, S, acsc, pi, simplify, solveset, symbols


@pytest.mark.parametrize(
    "target",
    [3*pi/2, 5*pi/2, -3*pi/2, 7*pi/2, 11*pi/2, -5*pi/2],
)
def test_solveset_acsc_filters_targets_outside_principal_range(target):
    x = symbols("x")
    sol = solveset(Eq(acsc(x), target), x, S.Complexes)

    assert all(simplify(acsc(candidate) - target) == 0 for candidate in sol)
```